

Question 16: Quick Sort on Nearly Sorted vs Random Datasets

Aim

To implement **Quick Sort**, measure its execution time on **nearly sorted** and **random** datasets, and analyze how different pivot selection methods affect performance.

Algorithm

Quick Sort Algorithm

1. Start with the input array.
2. Choose a pivot element.
3. Partition the array into two parts:
 1. Elements smaller than the pivot.
 2. Elements greater than the pivot.
4. Place the pivot in its correct position.
5. Recursively apply Quick Sort to the left subarray.
6. Recursively apply Quick Sort to the right subarray.
7. Continue until all elements are sorted.
8. Measure the execution time for the nearly sorted dataset.
9. Measure the execution time for the random dataset.
10. Compare the execution times.
11. Analyze how different pivot selection methods affect the performance.
12. Display and interpret the results.

Pseudocode

QUICKSORT(A, low, high)

IF low < high THEN

 pivot_index $\leftarrow$ PARTITION(A, low, high)

 QUICKSORT(A, low, pivot_index - 1)

 QUICKSORT(A, pivot_index + 1, high)

END IF

Partition using Last Element as Pivot

PARTITION(A, low, high)

 pivot $\leftarrow$ A[high]

 i $\leftarrow$ low - 1

```
FOR j  $\leftarrow$  low TO high - 1 DO
```

```
    IF  $A[j] \leq \text{pivot}$  THEN
```

```
         $i \leftarrow i + 1$ 
```

```
        SWAP  $A[i]$  and  $A[j]$ 
```

```
    END IF
```

```
END FOR
```

```
SWAP  $A[i + 1]$  and  $A[\text{high}]$ 
```

```
RETURN  $i + 1$ 
```

Python Program for Google Colab

Copy and run this entire program in **Google Colab**:

```
import random
```

```
import time
```

```
import sys
```

```
# Increase recursion limit
```

```
sys.setrecursionlimit(20000)
```

```
# -----
```

```
# QUICK SORT WITH LAST PIVOT
```

```
# -----
```

```
def partition_last(arr, low, high):
```

```
    pivot = arr[high]
```

```
    i = low - 1
```

```
    for j in range(low, high):
```

```
        if arr[j] <= pivot:
```

```
            i += 1
```

```
            arr[i], arr[j] = arr[j], arr[i]
```

```
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
```

```
    return i + 1
```

```
def quick_sort_last(arr, low, high):
```

```
    if low < high:
```

```
        pi = partition_last(arr, low, high)
```

```
        quick_sort_last(arr, low, pi - 1)
```

```

    quick_sort_last(arr, pi + 1, high)

# -----
# QUICK SORT WITH RANDOM PIVOT
# -----

def partition_random(arr, low, high):

    random_index = random.randint(low, high)

    # Move random pivot to end
    arr[random_index], arr[high] = arr[high], arr[random_index]

    return partition_last(arr, low, high)

def quick_sort_random(arr, low, high):

    if low < high:

        pi = partition_random(arr, low, high)

        quick_sort_random(arr, low, pi - 1)
        quick_sort_random(arr, pi + 1, high)

# -----
# CREATE NEARLY SORTED DATA
# -----

def create_nearly_sorted(n):

    arr = list(range(n))

    # Make 1% random swaps
    swaps = max(1, n // 100)

    for _ in range(swaps):

        i = random.randint(0, n - 1)
        j = random.randint(0, n - 1)

        arr[i], arr[j] = arr[j], arr[i]

    return arr

# -----
# TEST FUNCTION

```

```

# -----

def measure_time(data, pivot_type):

    arr = data.copy()

    start = time.perf_counter()

    if pivot_type == "Last Pivot":
        quick_sort_last(arr, 0, len(arr) - 1)

    elif pivot_type == "Random Pivot":
        quick_sort_random(arr, 0, len(arr) - 1)

    end = time.perf_counter()

    return end - start

# -----
# MAIN PROGRAM
# -----

sizes = [500, 1000, 2000, 5000]

print("QUICK SORT PERFORMANCE COMPARISON")
print("-" * 70)

print(f"{'Size':<10} {'Nearly Sorted (Last)':<25}"
      f"{'Random (Last)':<20}"
      f"{'Nearly Sorted (Random)':<25}"
      f"{'Random (Random)':<20}")

print("-" * 100)

for n in sizes:

    nearly_sorted = create_nearly_sorted(n)

    random_data = list(range(n))
    random.shuffle(random_data)

    # Measure times
    t1 = measure_time(nearly_sorted, "Last Pivot")
    t2 = measure_time(random_data, "Last Pivot")

    t3 = measure_time(nearly_sorted, "Random Pivot")
    t4 = measure_time(random_data, "Random Pivot")

```

```
print(f'{n:<10}'
      f'{t1:<25.6f}'
      f'{t2:<20.6f}'
      f'{t3:<25.6f}'
      f'{t4:<20.6f}')
```

5. Sample Input

Nearly Sorted Dataset

[1, 2, 3, 4, 7, 6, 5, 8, 9, 10]

Random Dataset

[7, 2, 9, 1, 5, 8, 3, 10, 4, 6]

Output

... QUICK SORT PERFORMANCE COMPARISON				
Size	Nearly Sorted (Last)	Random (Last)	Nearly Sorted (Random)	Random (Random)
500	0.004212	0.000521	0.000712	0.000709
1000	0.010864	0.001257	0.001709	0.002502
2000	0.019818	0.002794	0.003634	0.004651
5000	0.036301	0.007249	0.009557	0.009951

Manual Process

Consider:

[4, 2, 7, 1, 3]

Using the **last element as pivot**:

Pivot = 3

Partition the elements:

Smaller than 3: [2, 1]

Pivot: [3]

Greater than 3: [4, 7]

So:

[2, 1] + [3] + [4, 7]

Now sort [2, 1]:

Pivot = 1

[1, 2]

Sort [4, 7]:

Pivot = 7

[4, 7]

Pivot Selection Impact

Pivot Method	Nearly Sorted Data	Random Data	Performance
First/Last Element	Can be very slow	Usually good	May reach $O(n^2)$
Random Pivot	Good	Good	Average $O(n \log n)$
Middle Element	Better	Good	Usually efficient
Median-of-Three	Very good	Very good	Reduces bad partitions

Techniques to Improve Efficiency

1. Random Pivot

Choose a random element as the pivot to avoid predictable worst cases.

2. Median-of-Three Pivot

Choose the median of:

First + Middle + Last

This often produces a better balanced partition.

3. Hybrid Quick Sort

Use **Insertion Sort** for very small subarrays.

4. Detect Bad Partitions

If partitions become repeatedly unbalanced, switch to another sorting algorithm.

5. Introspective Sort

Use Quick Sort initially and switch to Heap Sort when recursion becomes too deep.

Conclusion

Quick Sort performance strongly depends on **pivot selection**. On nearly sorted data, choosing the first or last element as the pivot may cause unbalanced partitions and degrade performance to **$O(n^2)$** . Random or median-based pivot selection creates more balanced partitions and improves efficiency, generally maintaining **$O(n \log n)$** performance. Therefore, better pivot selection techniques make Quick Sort more efficient and reliable.